

ВВЕДЕНИЕ В ПРОГРАММИРОВАНИЕ ВЫЧИСЛЕНИЙ НА GPU С ИСПОЛЬЗОВАНИЕМ OPENCL

ЕГОР ОРАЧЕВ
RENDER PROGRAMMER
SABER INTERACTIVE

5 МАЯ, 2026

О себе

- **Lead Render Engineer**
- 4+ лет работаю в **Saber Interactive**
- 6+ лет опыта работы с GPU и графикой
- Работал над проектами
 - Warhammer 40K: Space Marine 2**
 - Roadcraft**
 - Toxic Command**
- Бакалавриат и магистратура Мат-Меха **СПбГУ**

Структура презентации

Часть I.

Теоретическая основа

Часть II.

Практический пример

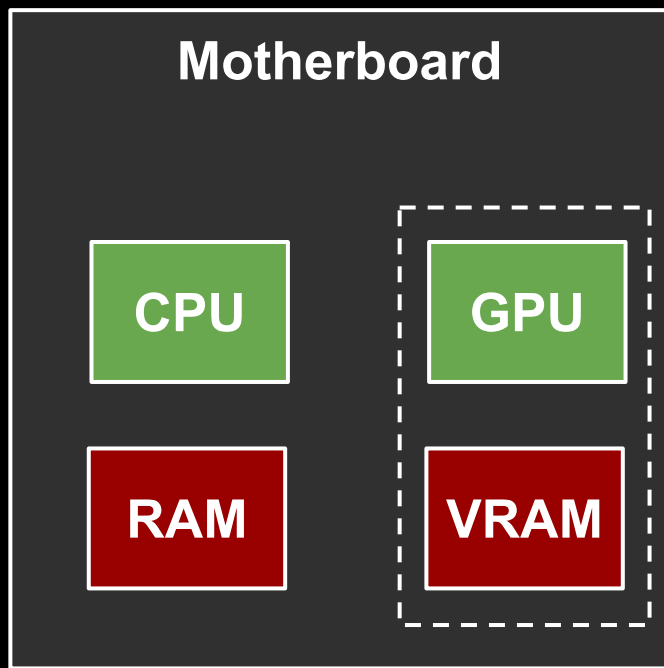
Часть III.

О геймдев специфике

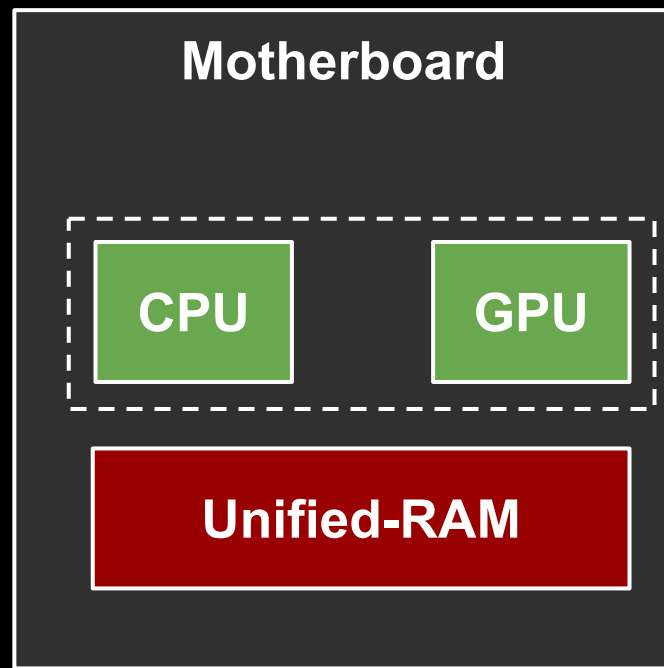
Часть I: Теоретическая основа

- Краткая история развития GPU
- Что такое GPGPU-вычисления
- Введение в OpenCL
- Примеры программ на OpenCL
- Современные архитектуры
- Проблемы производительности в GPU
- Разреженная линейная алгебра на GPU
- Стандарт GraphBLAS и проект spla

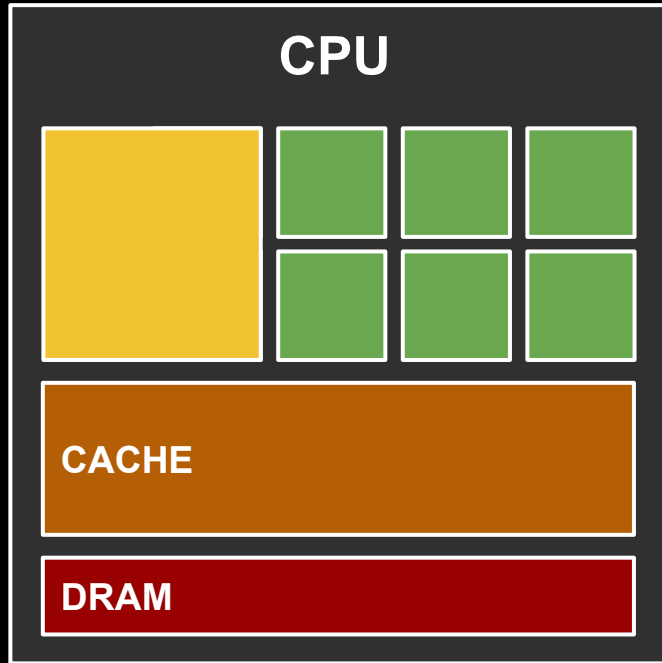
Что такое GPU?



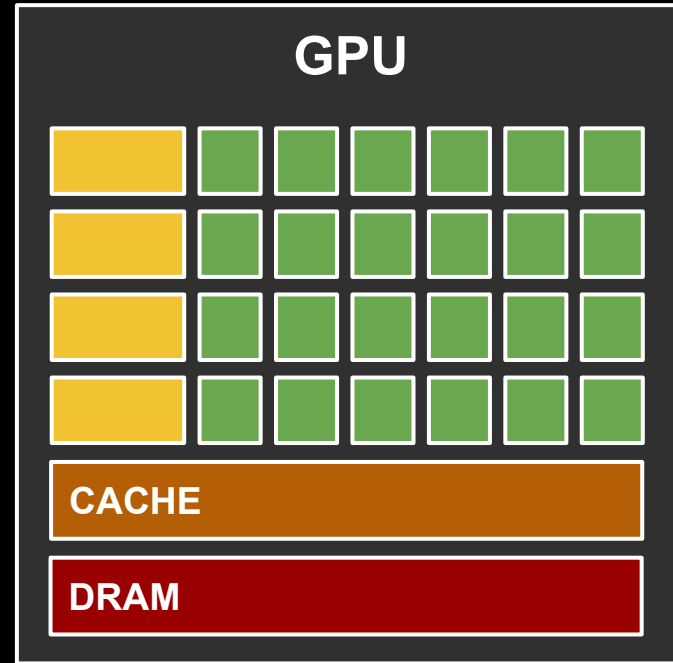
/



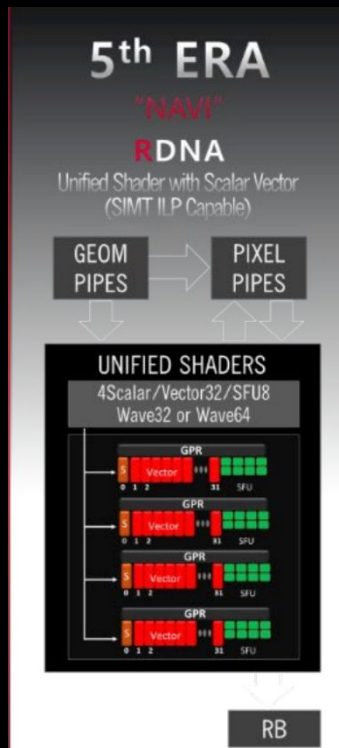
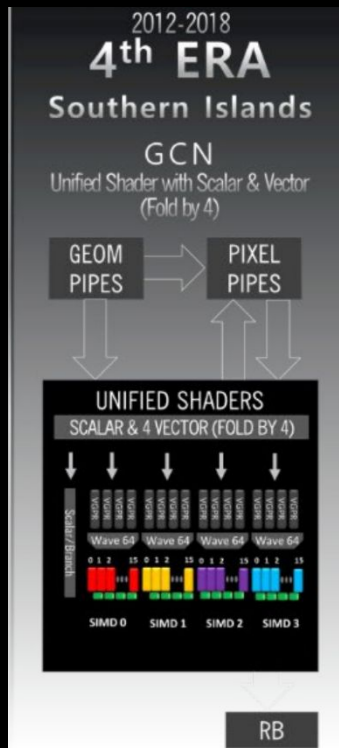
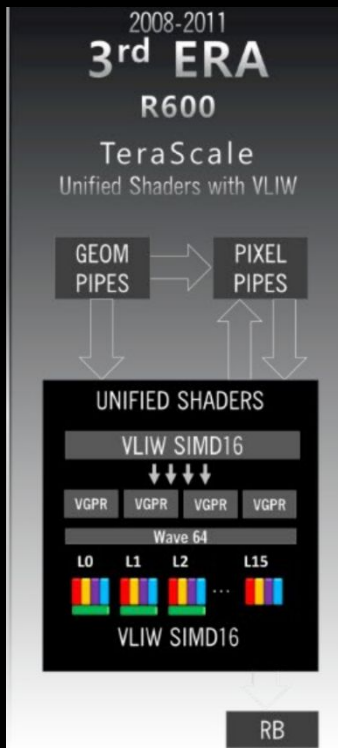
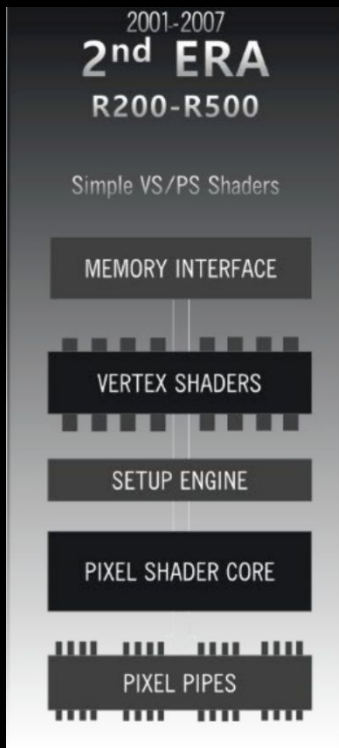
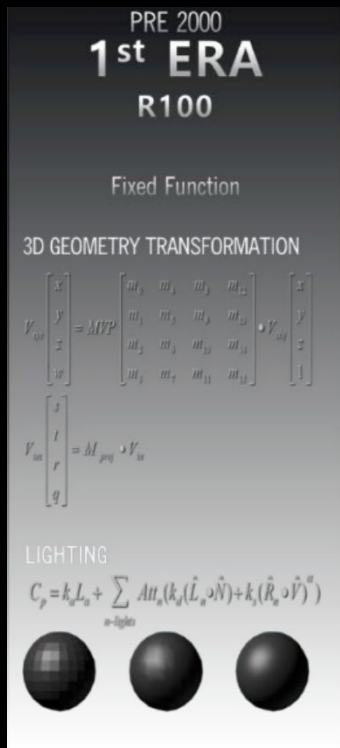
CPU vs GPU



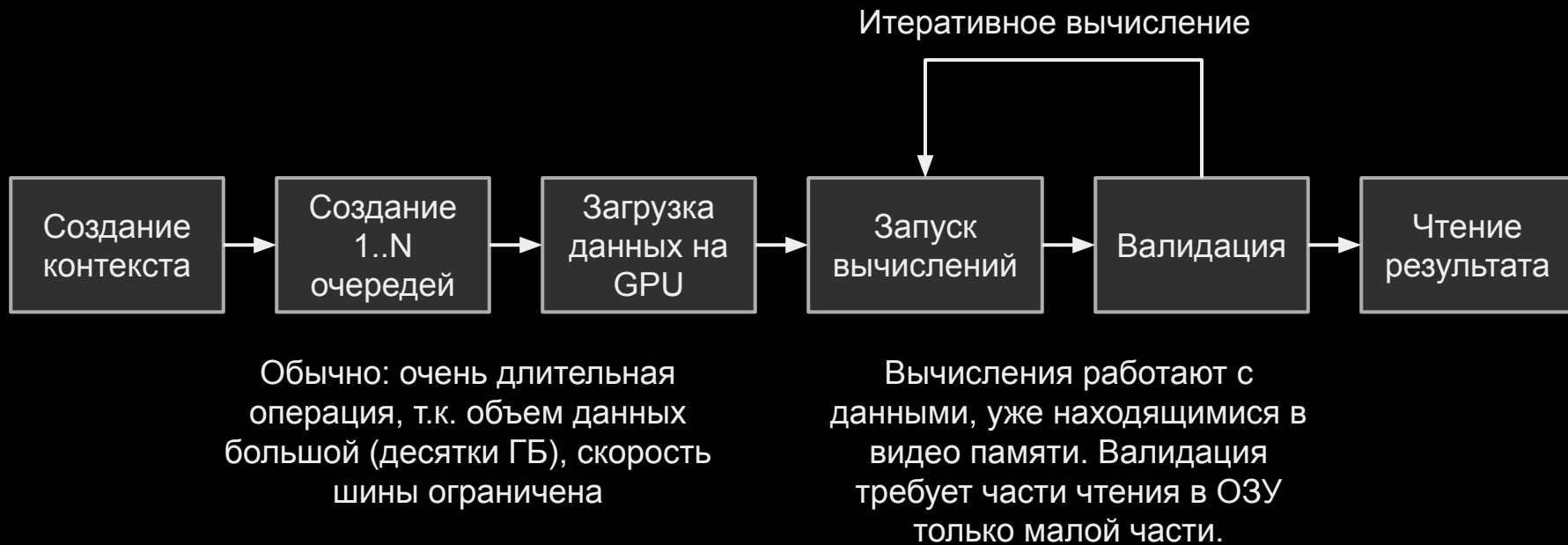
VS



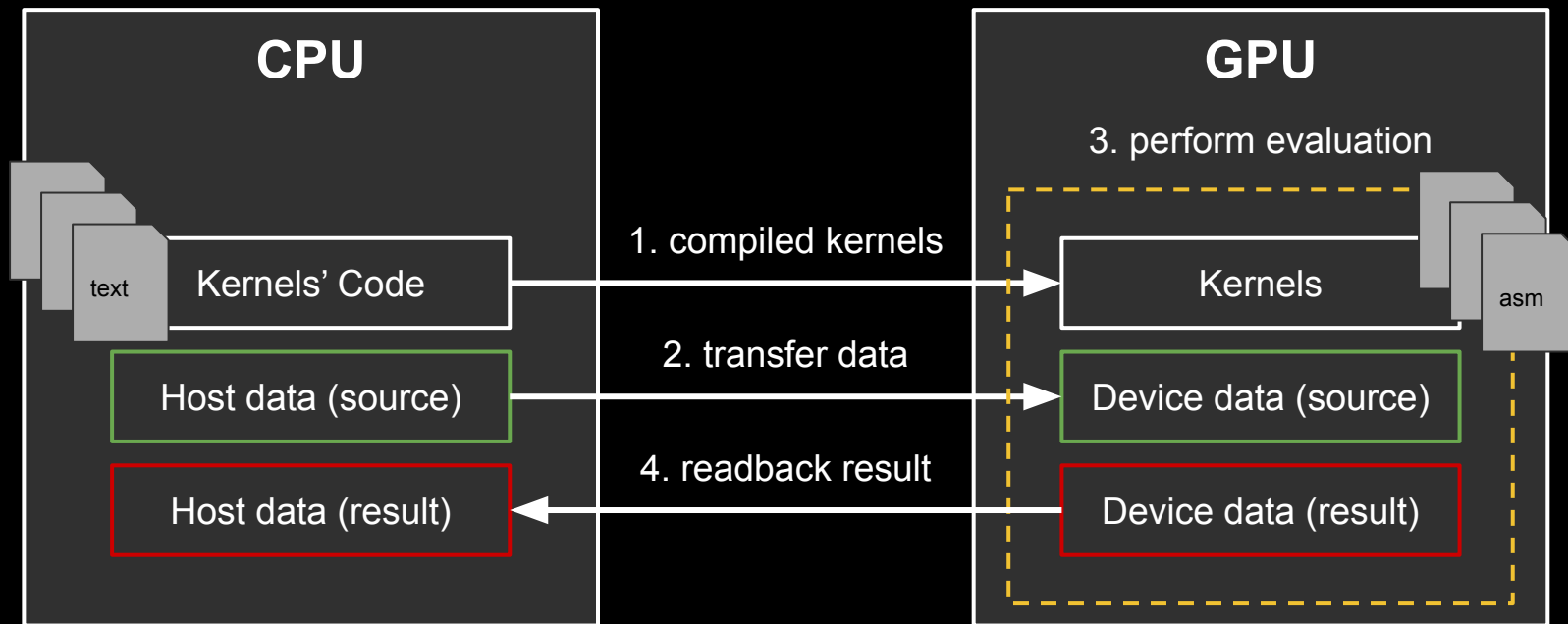
Краткая история эволюции GPU



Типовое GPU приложение



GP GPU-вычисления



Timeline вычислений

CPU

Компиляция
GPU ядер

Подготовка
данных

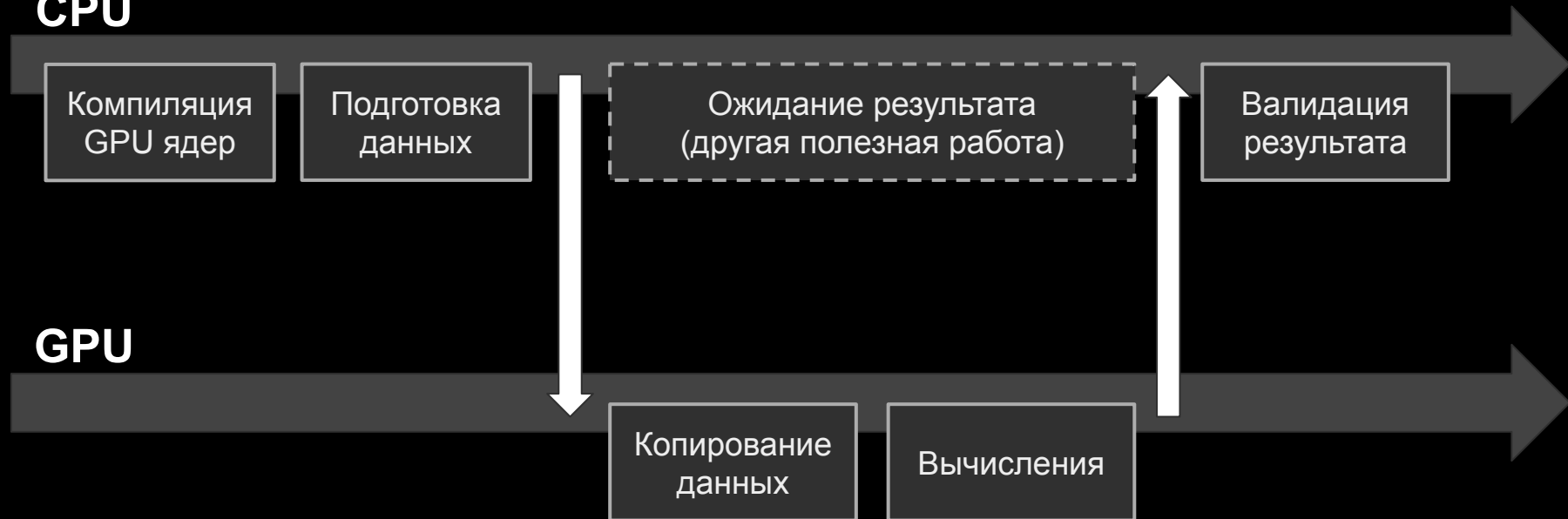
Ожидание результата
(другая полезная работа)

Валидация
результата

GPU

Копирование
данных

Вычисления



API для работы

- Графические вычисления

- Vulkan
- Direct3D
- Metal
- GNM / GNMX
- OpenGL

→ Графические приложения
Пользовательский интерфейс
Видеоигры

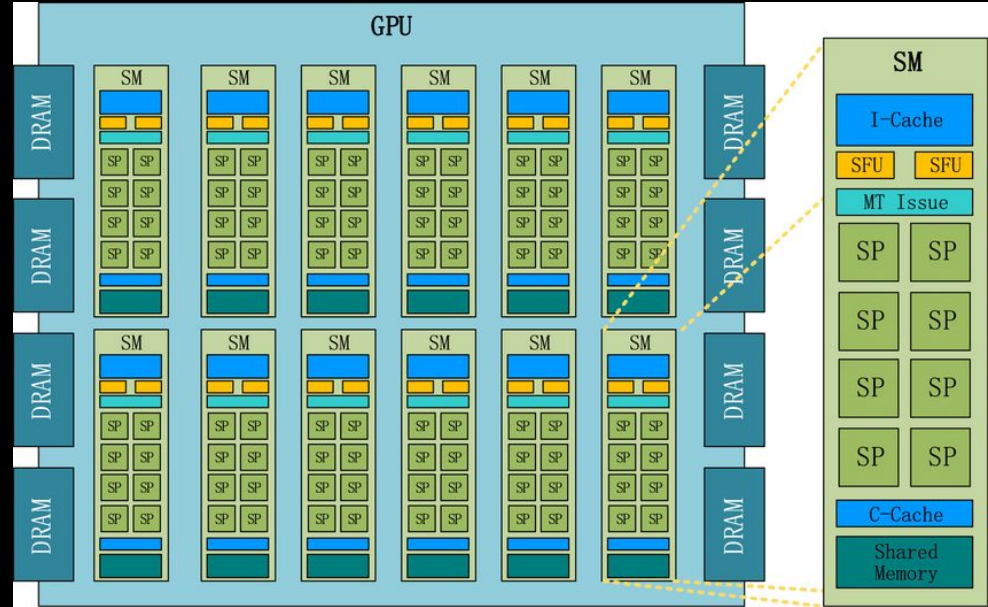
- Неграфические вычисления

- CUDA
- OpenCL
- SyCL

→ Математическое ПО
Машинное обучение
Анализ данных

CUDA

- Compute unified device architecture
- Платформа параллельных вычислений
- Промышленный API
- Язык, модель, набор инструментов, библиотеки, компиляторы, etc.



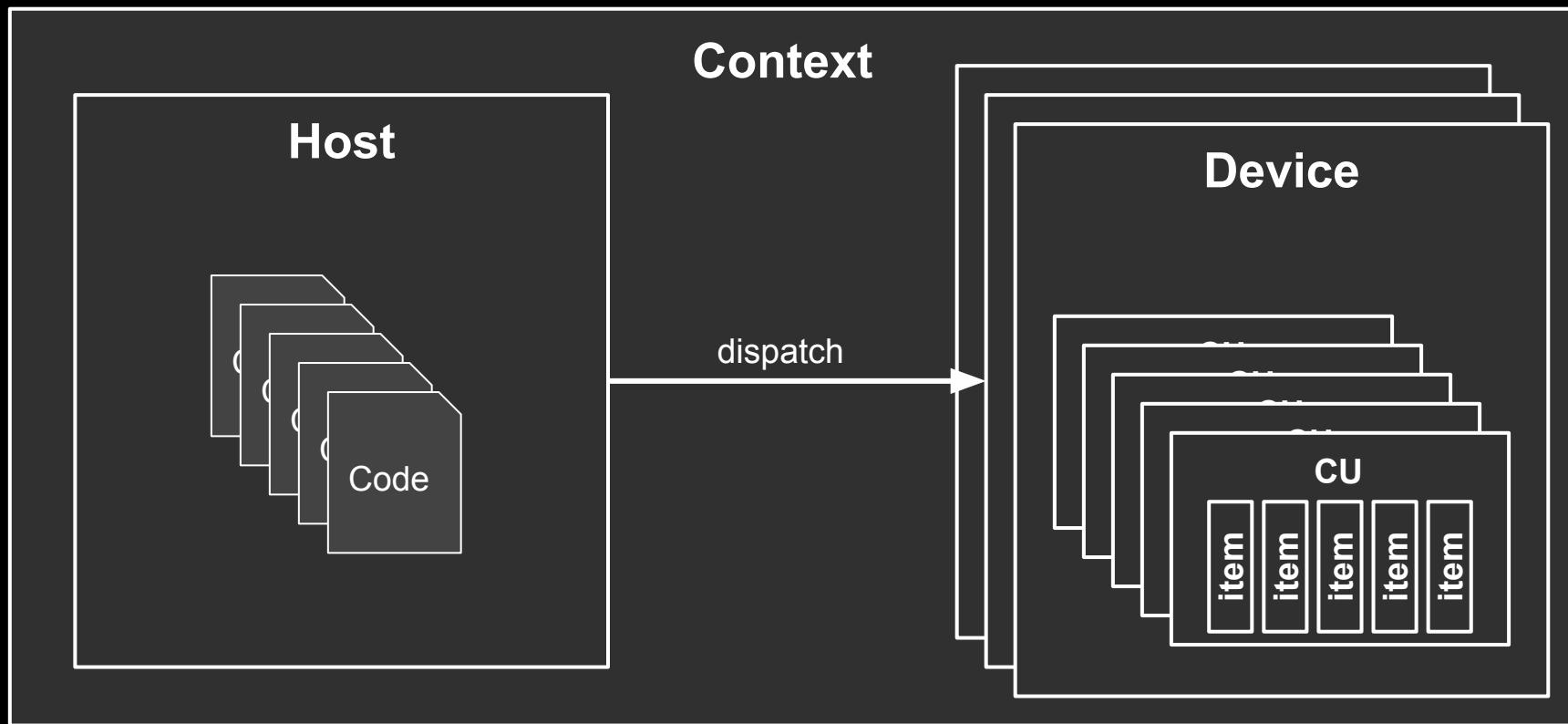
OpenCL

- Open Computing Language
- Фреймворк для написания компьютерных программ, связанных с параллельными вычислениями на различных графических и центральных процессорах, а также FPGA
- Включает язык программирования, и интерфейс программирования приложений
- Обеспечивает параллелизм на уровне инструкций и на уровне данных и является осуществлением техники GPGPU
- Является полностью открытым стандартом
- Доступен на ускорителях Intel, Nvidia, AMD

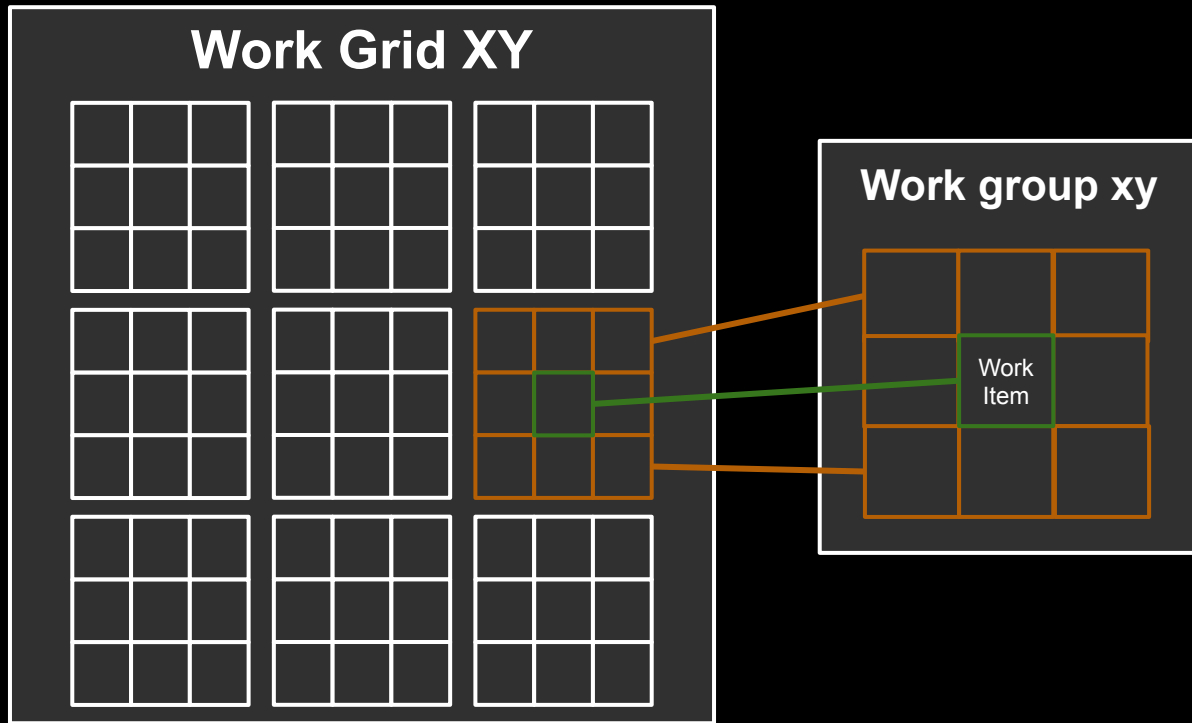
Модель OpenCL

- **Platform**
Устройства и контекст выполнения
- **Memory**
Иерархия памяти, видимость операций чтения/записи, кеш
- **Execution**
Выполнение программ на GPU и синхронизация
- **Programming**
Языковые и программные инструменты для работы

Platform model

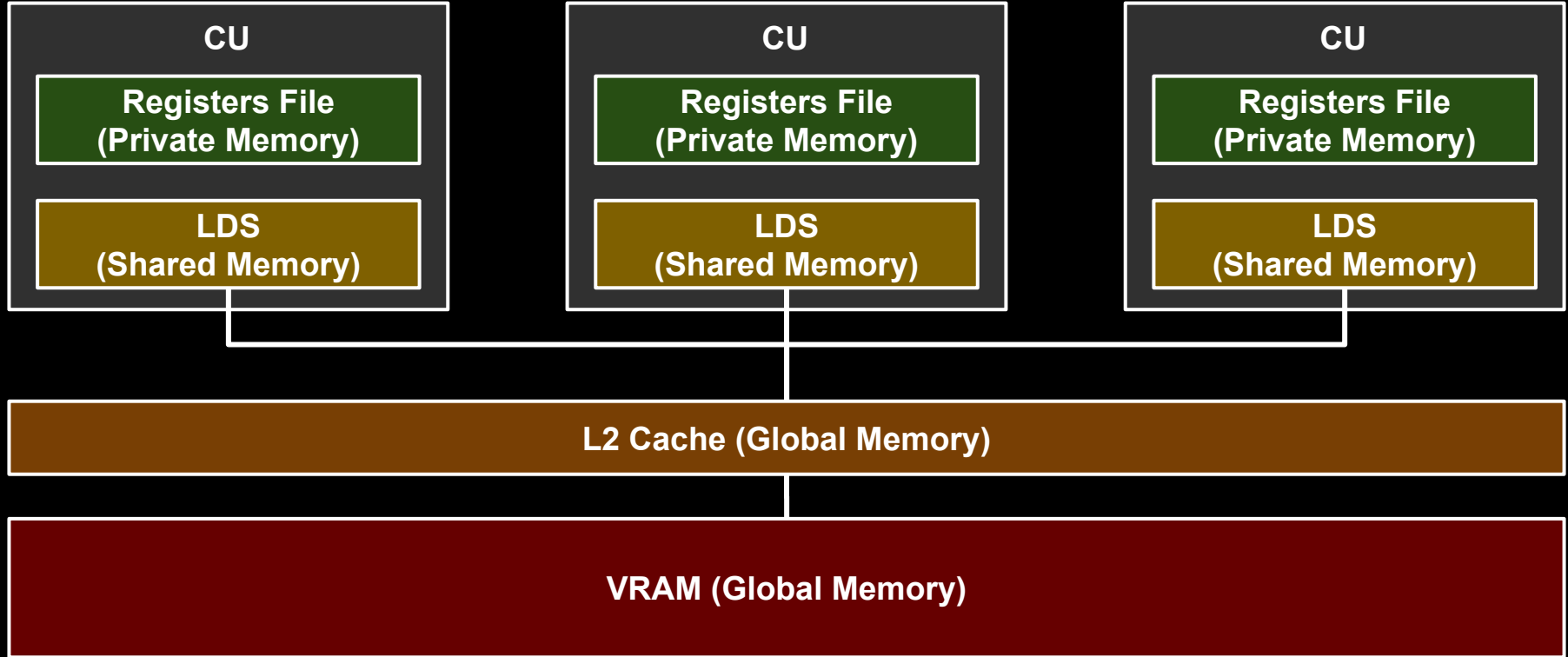


Execution model



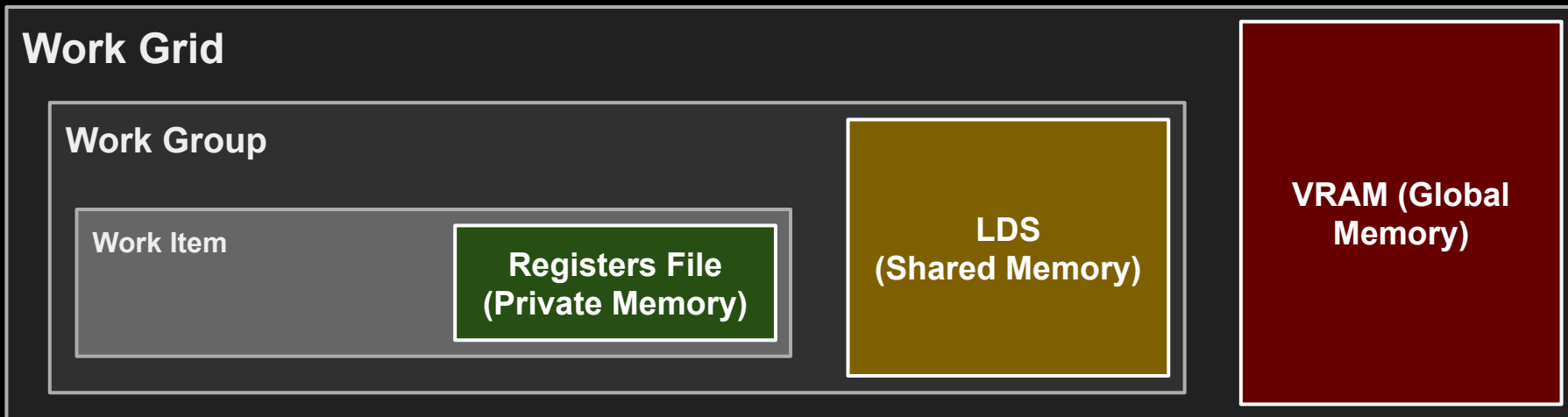
- Total items
 $X * Y$
- Group size
 $x * y$
- Group count
 $(X / x) * (Y / y)$
- В примере
Сетка 9x9
Размер группы 3x3

Memory model



Memory model

- Одному **Work Item** доступны его приватные регистры
- Все потоки внутри **Work Group** могут иметь общую Shared Memory
- Всем **Work Group**'ам доступна общая глобальная память



Programming model

- Task-parallel processing
- Выполнение фиксированной функции над 1 элементом
- Запуск сетки потоков для параллельной обработки всех элементов
- Определение программы (ядра) как процедуры на C подобном языке, которая вызывается для каждого work item

```
__kernel void my_kernel(__global const int* a, __global const int* b, __global int* c) {  
    size_t idxInGrid = get_global_id(0); // Item id in grid  
    size_t idxInGroup = get_local_id(0); // Item id in group  
    int x = a[idxInGrid]; // Load from global memory  
    __shared int d[GROUP_SIZE]; // Declare shared array, visible for all items inside our group  
    . . .  
}
```

Базовые определения

- Platform → Платформа
- Device → Устройство
- Context → Контекст
- Buffer → Буфер (регион памяти)
- Program → Программа
- Kernel → Ядро
- CommandQueue → Очередь команд
- Dispatch → Запрос на исполнение
- NDRange → N-мерный регион

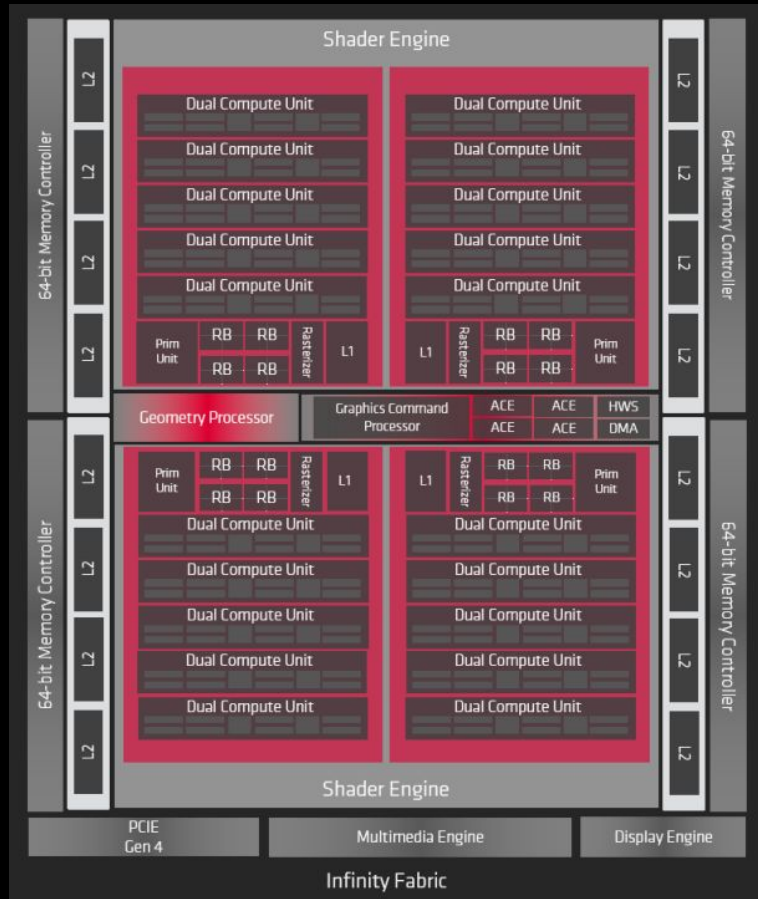
Архитектуры

- Nvidia Pascal
- Nvidia Turing
- Nvidia Ampere
- Nvidia Ada Lovelace
- AMD GCN (Graphics Core Next)
- AMD RDNA (Radeon DNA)
- AMD RDNA-2

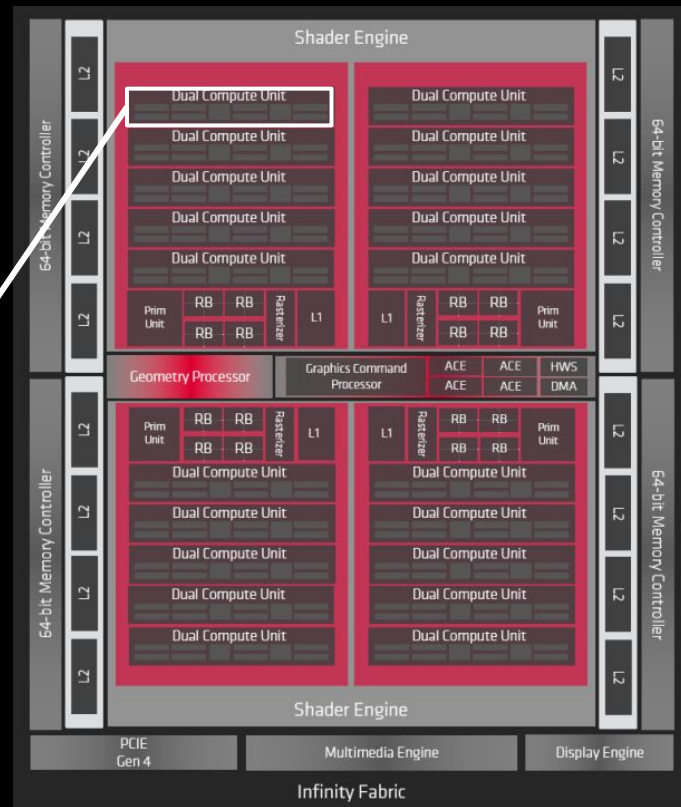
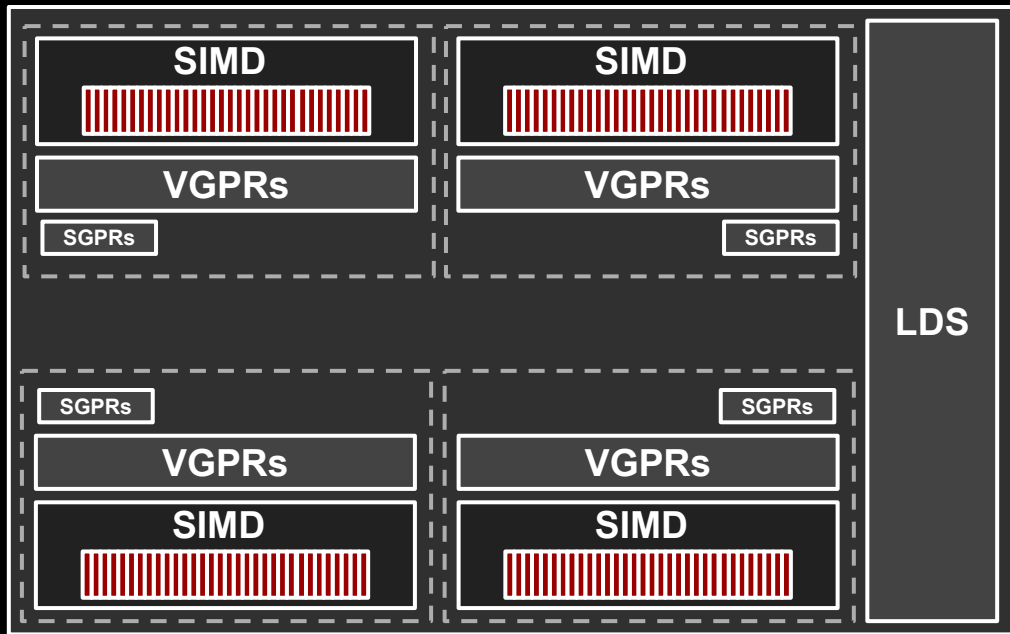


RDNA

- GPU состоит из набора CU
- CU имеет общий LDS
- Workgroup выполняется на одном CU
- CU (Compute Unit) состоит из набора SIMD (32 ALU) процессоров
- SIMD процессор исполняет 1 *wave32* за 1 такт процессора
- SIMD процессор имеет фиксированный набор VGPR и SGPR регистров



RDNA Compute Unit



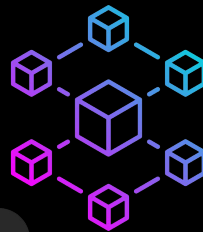
Nvidia GPUs Architecture

- GPU состоит из набора SM
- SM имеет общий L1 кэш
- SM (streaming multiprocessor) состоит из набора CUDA cores
- CUDA cores объединены в группы *warp* по 32 по принципу SIMD
- 1 SIMD группа выполняется за 1 такт
- Workgroup выполняется на одном SM процессоре



Практическое применение

- Игровые движки / видеоигры
- Графические редакторы
- Видеомонтаж / фоторедактирование
- UI фреймворки
- Машинное обучение
- Нейронные сети
- Анализ данных
- Блокчейн



Часть II: Практический пример

- См. `ipython-notebook`
- Постановка задачи
- Данные для анализа
- Анализ различными средствами

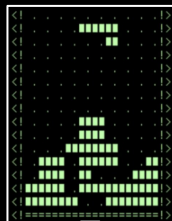
Идеи для экспериментов

- Подключить **cugraph** и запустить bfs на **Cuda**
Нужна **Nvidia GPU**
- Реализовать **bfs layers** используя **OpenCL**
Добавить новый kernel и сделать функцию обвязки
- Реализовать **bfs tree** используя **OpenCL**
Добавить новый kernel и сделать функцию обвязки
- Доработать генератор графов для экспериментов
Настройка плотности, максимальной глубины, оптимизация скорости
- Собрать стенд и сравнить **naive**, **scipy**, **opencl**, **cugraph**
Подобрать датасет, сделать замер времени/памяти, вывести таблицу и график

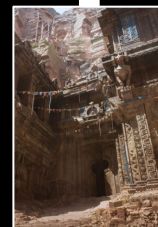
Часть III: О геймдев специфике

- Краткая историческая справка
- Чем занимается `render engineer`
- Тренды и тенденции
- Нюансы и стереотипы

Историческая справка



THEN



NOW

Задачи графики

- Рисование 3D геометрии
- Расчет освещения
- Симуляция большого числа частиц / VFX
- Симуляция одежды
- Симуляция ветра
- Экранные пост-эффекты
- Симуляция облаков
- Динамическая смена дня и ночи
- Расчет теней
- Рассадка травы



Примеры



Тренды и тенденции

- Развитие геймдева в азии
- Развитие мобильного направления
- Автоматизация разработки с использованием AI
- Сокращение рынка вакансий
- Высокая конкуренция
- Спад AAA-направления
- Затухание VR направления

Нюансы и стереотипы

- “Игры это для детей”
- “Нет перспектив роста”
- “Нет интересных и сложных задач”
- “Мало платят”
- “Постоянные переработки”
- “Токсичная атмосфера в компании”
- “Сложно найти хорошую компанию”
- “Есть пространство для творчества”

Вывод: если область **очень** нравится, то оно стоит того, иначе – **нет**